SMARTMENU

AI Usage Report

HackIndia AI-First Startup Hackathon 2026

Prepared: 30 October 2026

Project purpose
A multi-restaurant SaaS platform for managing and sharing mobile-friendly digital menus through QR codes.

Prepared from the project development context available as of the preparation date. Confirm the tool list, prompt summaries, and final tests with the team before submission.

1. Project Overview

SmartMenu is a multi-restaurant SaaS concept for creating and managing digital restaurant menus. Restaurant owners can manage restaurant information, menu categories and dishes, add dish images, and generate a QR code that customers can scan to open a mobile-friendly menu.

The current product includes an owner dashboard, menu management, QR Code Studio, a customer-facing menu with search and dietary filters, dish details, menu-engagement analytics, and a rule-based insights feature. The current scope does not include online ordering, payment processing, or verified sales/revenue analytics.

2. AI Tools and Supporting Technologies

Tool / technology	Role in the project
Google AI Studio (Build mode)	AI-assisted application development, code generation and modification, UI implementation, debugging, and iterative refinement of the web application.
ChatGPT	Planning and product clarification, feature breakdowns, implementation prompts, debugging guidance, test checklists, documentation, and submission preparation.
Supabase	Backend services used for authentication, database storage, and dish-image storage. Supabase is a project technology, not itself an AI model.
GitHub	Version-control hosting and source-code backup; a separate private repository was created and synced. The HackIndia-assigned repository must be checked separately for final submission.
Browser / local testing	Manual verification of app flows, mobile layout, image behavior, and interactions, plus available TypeScript/build checks.

This report describes AI-assisted development. It does not claim that every line of code was written by AI or that the application uses a generative AI model at runtime for every feature.

3. AI-Assisted Development Workflow

1. Requirements and product planning: clarify the problem, target users, SaaS approach, initial scope, and the difference between restaurant-owner workflows and customer menu browsing.

2. Interface and feature implementation: use AI-assisted prompts to create and refine dashboard screens, menu management, QR tools, customer menu interactions, and responsive layouts.

3. Backend integration: configure Supabase tables and access policies for restaurants, restaurant members, categories, dishes, and menu analytics events. Configure Storage for dish images and access policies intended to restrict uploads and changes to authorized restaurant owners/managers.

4. Authentication: implement a sign-in/create-account interface connected to Supabase authentication, with account status and sign-out behavior.

5. Reliability and fallback handling: refine image upload handling so local preview/fallback behavior is available when cloud configuration or upload fails, while cloud uploads use a valid restaurant UUID and supported image formats.

6. Testing and refinement: use build/type-check results and manual checks to identify and fix layout, upload, persistence, permissions, and interaction issues.

7. Documentation and delivery preparation: use AI assistance to prepare testing checklists, repository guidance, pitch-deck content, and this usage report.

4. Representative AI Prompt Summaries

The entries below are representative summaries of development requests discussed during the project. They are not presented as a verbatim export of every prompt or as a complete prompt-history log.

Task	Representative prompt summary	Review / validation
Product scope	Plan a free-first, multi-restaurant digital-menu SaaS rather than a single-restaurant hardcoded demo; keep ordering and payments out of the initial scope.	Compared features against the intended hackathon product and clarified unsupported claims.
Dashboard and customer UI	Build/refine a mobile-friendly restaurant dashboard, menu-management screens, QR tools, and a customer-facing menu with search and dietary filters.	Opened screens in the app and checked navigation, visual layout, and mobile overflow.
Supabase schema and security	Set up restaurant, membership, category, dish, and analytics tables with tenant-aware RLS policies; avoid exposing privileged keys.	Reviewed intended roles and policies; applied SQL in the Supabase project and ran available checks.
Dish-image upload	Validate image types and size, use a restaurant-specific cloud storage path, and preserve a local fallback when cloud upload is unavailable.	Tested image upload and visibility; checked TypeScript/build results.
Authentication	Add sign-in/create-account access to Settings using the existing Supabase auth integration, with account status and sign-out.	Tested sign-in; checked build/type-check and available audit results.
Analytics accuracy	Implement local/free rule-based menu engagement insights and avoid claiming orders, sales, or revenue without ordering functionality.	Checked labels and interpretation against actual interaction-based analytics.
Responsive fixes	Fix mobile overflow and card/grid layout problems without unnecessarily changing the existing visual theme.	Retested the relevant layouts and build.
Documentation and submission	Prepare a feature checklist, repository guidance, demo flow, pitch-deck outline, and an AI usage report for the hackathon.	Reviewed deliverables against the known event checklist; final submission requirements should be confirmed in the team dashboard.

5. Human Review and Team Contributions

The project team selected the SmartMenu concept and its intended users, decided which features belonged in the initial version, and directed iterative changes based on the desired product behavior and visual design.

Team members used the development environment, Supabase dashboard, GitHub, and the published application to configure services, apply changes, test workflows, and maintain a source-code backup. AI-generated code and suggestions require human review; build success alone does not prove that every security rule or user workflow is correct.

Before submission, the team should verify the final public/organizer-accessible repository, live application URL, demo video, pitch deck, and any organizer-specific forms or instructions.

6. AI at Development Time vs. AI at Runtime

AI-assisted development: Google AI Studio and ChatGPT were used to help plan, generate, modify, explain, debug, and document application code and features.

Rule-based runtime insights: the current AI Menu Insights feature is described in the project as a rule-based analytics engine that derives insights from available menu-engagement data. It should not be represented as a live

generative-AI feature unless the implementation is later changed and verified.

Other application services: Supabase provides authentication, database, and image-storage capabilities. These services are backend infrastructure, not evidence that a generative AI model is running inside the product.

No claims are made here about a production LLM API, model name, model accuracy, customer adoption, revenue, or business outcomes because these were not verified as part of the current implementation.

7. Testing and Validation Summary

Project development notes recorded successful TypeScript checks and production builds at multiple refinement points. A previously reported live audit also checked public restaurant retrieval, category/dish retrieval, anonymous analytics insertion, unpublished-restaurant access restrictions, and India Standard Time timestamp formatting.

Image handling was refined for JPEG, PNG, and WebP files up to 5 MB, with restaurant-specific cloud paths and fallback behavior. The Storage bucket and related policies were configured to allow public image reads while restricting writes to authorized owner/manager roles.

These results describe checks reported during development, not an independent security audit or a guarantee of complete production security. The team should repeat the final tests against the exact published build and confirm the RLS/Storage policies before judging or public release.

8. Limitations and Responsible Disclosure

The current application is a digital-menu and engagement product, not a point-of-sale system. It does not currently substantiate claims about completed orders, payment processing, sales, or revenue.

AI Menu Insights uses rule-based logic. Do not label it as a live generative AI model feature.

Local/browser fallback data may not synchronize across devices. Cloud synchronization depends on correct Supabase configuration and permissions.

The team should not commit Supabase service-role keys, database passwords, private API keys, or other secrets to GitHub. The final repository should be inspected before submission.

Any feature, test, tool, or prompt not actually used should be removed or corrected before the report is submitted. Add the names of all teammates and any additional AI tools only after confirming them.

9. Conclusion

AI tools supported the SmartMenu team's development process by accelerating planning, UI implementation, code refinement, debugging, testing guidance, and documentation. The team remains responsible for verifying the implementation, ensuring truthful product claims, and validating security and functionality before submission.

Prepared for: HackIndia AI-First Startup Hackathon 2026